Universidad del Valle de Guatemala Programación Orientada a Objetos
Segundo Ciclo, 2026
Catedrático: Kevin Antonio Velasquez Aguilar



Lab 3 - Herencia y polimorfismo

Tecnología Médica

Obed Natanael Tot Paiz
Carné:26971

Fecha de entrega: 18/10/2026

1. ANÁLISIS

1.1. ¿Qué problema debemos resolver?

El hospital tiene citas duplicadas, asignaciones incorrectas de personal y dificultades para reorganizar los horarios y el sistema debe centralizar la información de los trabajadores y las citas para evitar conflictos, facilitar los reagendamientos y calcular los salarios según el tipo de trabajador.

El usuario principal será el administrador del hospital, quien registra personal, administra citas y consulta reportes y el enunciado no especifica que pacientes o trabajadores inicien sesión.

1.2. ¿Qué funciones debe tener?

Área	Funciones requeridas
Personal médico	Registrar trabajadores de al menos cuatro tipos y mostrar su información.
Salarios	Calcular el salario de cada trabajador según su especialidad y obtener totales por departamento.
Citas	Registrar paciente, trabajador, fecha, hora, tipo y estado. Consultar citas por estado y trabajador.
Asignación	Buscar personal compatible con el tipo de cita y disponible en el horario solicitado.
Reagendamiento	Detectar conflictos, buscar una alternativa, actualizar la cita y conservar el historial.
Notificaciones	Informar los cambios mediante mensajes en consola.
Utilización	Mostrar qué tanto se aprovecha la disponibilidad del personal.

1. 3. ¿Qué información debemos manejar?

Podemos identificar estas entidades iniciales, es una propuesta para organizar el análisis y después la convertiremos en clases. El enunciado únicamente solicita el nombre del paciente.

Entidad	Información principal	Responsabilidad
Trabajador médico	ID, nombre, departamento, experiencia y salario base	Representar los datos comunes del personal.
Tipos de trabajador	Atributos propios de cada especialidad	Definir su información particular y cálculo salarial.
Cita médica	ID, nombre del paciente, trabajador, fecha, hora, tipo y estado	Representar una atención programada.
Cambio de agenda	Cita afectada, fecha anterior y nueva, trabajador anterior y nuevo, motivo y momento del cambio	Conservar el historial de reagendamientos.
Disponibilidad	Jornada laboral y ausencias del trabajador	Permitir comprobar si puede atender una cita.

1.4. ¿Cómo se aplicaría la herencia?

Todos los trabajadores comparten información, pero tienen características y cálculos salariales diferentes y por eso se debe proponer una clase base abstracta TrabajadorMedico y comenzar con los primeros cuatro tipos permitidos.

Subclase	Datos específicos	Cálculo salarial
DoctorGeneral	Especialización, capacidad diaria y tarifa de consulta	Base + consultas realizadas × tarifa
Cirujano	Operaciones que realiza, horas disponibles y bono por riesgo	Base + horas realizadas × tarifa por hora + bono

Enfermero	Turno y certificación	Base + bonificación nocturna, cuando corresponda
Radiologo	Equipos certificados y tarifa por estudio	Base + comisiones por estudios realizados

El enunciado usa algunos datos en las fórmulas, pero no los incluye en la lista inicial y por eso se debe agregar al diseño, la cantidad de consultas realizadas, las horas de cirugía trabajadas, el pago por hora, el bono nocturno y las comisiones. También debemos distinguir las horas del cirujano y las horas disponibles sirven para programar citas y las horas trabajadas: sirven para calcular cuánto se le debe pagar.

1. **5. ¿Dónde aparece el polimorfismo?**

La clase base puede declarar un comportamiento como calcularSalario(), que cada subclase implementará con su fórmula. Así, el administrador del hospital puede recorrer una colección de TrabajadorMedico y solicitar el salario de cada elemento. Cada objeto responderá según su tipo concreto.

También puede aplicarse al mostrar información mediante toString() y, como decisión de diseño, al consultar si un trabajador puede atender determinado tipo de cita. Una aplicación posible sería, reagendar una cita indicando una nueva fecha y hora y reagendar una cita indicando una nueva fecha, hora y trabajador. Esto es diferente de la sobrescritura y utilizaremos cuando cada subclase implemente su propio cálculo salarial.

1. 6. ¿Qué reglas necesitamos definir?

Aspecto	Regla propuesta
Identificadores	No permitir IDs repetidos de trabajadores ni de citas.
Compatibilidad	Asignar únicamente trabajadores capacitados para el tipo de atención.

Jornada	Programar citas dentro del horario del trabajador y fuera de sus ausencias.
Duración	Cada cita tendrá una duración para poder detectar superposiciones.
Conflictos	Un trabajador no podrá atender dos citas cuyos intervalos se superpongan.
Capacidad	Respetar el límite diario del doctor y las horas disponibles del cirujano.
Reagendamiento	Buscar primero otro horario del mismo trabajador; después, otro trabajador compatible.
Sin disponibilidad	Informar que no se encontró alternativa y conservar la cita original.
Historial	Guardar cada cambio sin eliminar los anteriores.
Nómina	Calcular la actividad realizada dentro de un período definido, por ejemplo, un mes.

La duración es especialmente importante ya que no basta con comprobar que dos citas tengan distinta hora de inicio. Una cita de 9:00 a 10:00 entra en conflicto con otra de 9:30 a 10:30. También debemos aclarar qué citas puede atender enfermería, porque los tipos de cita enumerados no incluyen una atención específica de enfermería y no conviene asignarle consultas o cirugías automáticamente sin definir esa regla.

1. 7. ¿Cómo manejaríamos los estados?

El laboratorio establece seis estados

PROGRAMADA, CONFIRMADA, EN_PROGRESO, COMPLETADA, CANCELADA y REAGENDADA.

Como propuesta:

- Una cita nueva comienza como PROGRAMADA.
- Al confirmar la asistencia pasa a CONFIRMADA.
- Al iniciar la atención pasa a EN_PROGRESO.
- Al terminar pasa a COMPLETADA.
- Si se anula, pasa a CANCELADA.
- Si cambia de horario o trabajador, pasa a REAGENDADA y puede confirmarse nuevamente.
- Las citas REAGENDADA deben seguir ocupando su nuevo horario. Las canceladas liberan el espacio. Las completadas y canceladas se conservan para consultas e historial.

1. **8. ¿Cómo se separan las responsabilidades con MVC?**

Componente	Responsabilidad
Modelo	Trabajadores, citas, disponibilidad, historial y reglas del negocio.
Vista	Mostrar el menú, solicitar datos y presentar resultados en consola.
Controlador	Recibir las acciones del usuario y coordinar las operaciones del modelo.

HospitalManager puede funcionar como un servicio del modelo que coordine asignaciones, reagendamientos y reportes, las fórmulas salariales deben permanecer en las clases de trabajadores correspondientes.

1. **9. ¿Qué debemos poder demostrar?**

Los escenarios

- Al menos 15 trabajadores de diferentes tipos.

- 20 citas en un día para probar disponibilidad y conflictos.
- Una ausencia que obligue a redistribuir citas.
- El cálculo mensual de la nómina.
- Una cirugía urgente que requiera mover otras citas.

Para el reporte de utilización:

Utilización (%) = minutos ocupados en citas activas ÷ minutos disponibles × 100

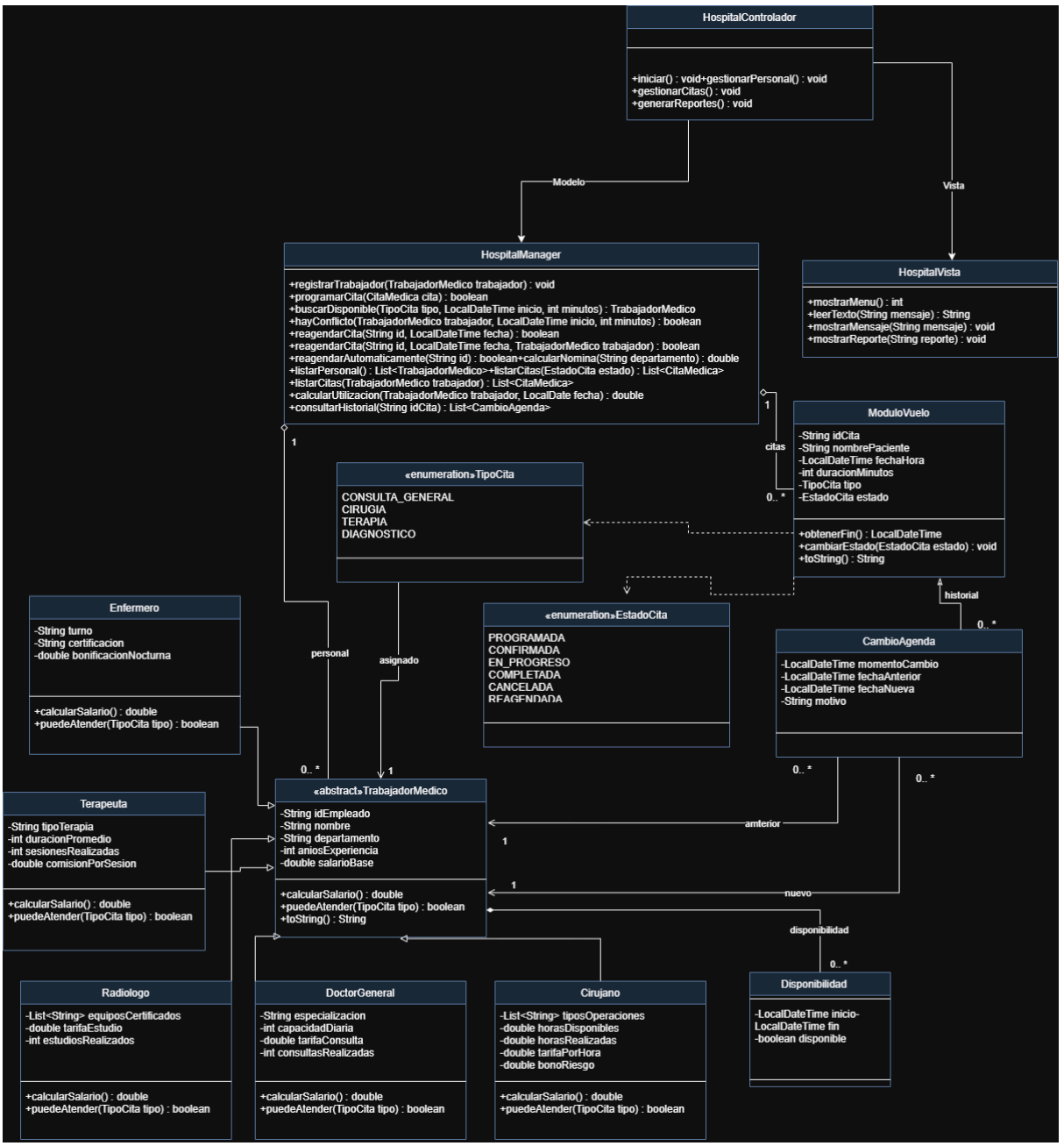
El cálculo debe corresponder al mismo período para todos los trabajadores.

1.10. Tipos de procesos

- Secuencial ejecuta instrucciones en orden. Ejemplo: calcular el pago de una consulta y generar el recibo.
- Condicional toma decisiones según una condición. Ejemplo: asignar una cita si el médico está disponible.
- Repetitivo ejecuta instrucciones varias veces. Ejemplo: recorrer la lista de trabajadores para calcular la nómina del hospital.
- El sistema utilizará herencia mediante una clase base TrabajadorMedico, de la cual derivarán DoctorGeneral, Cirujano, Enfermero y Radiologo. Se aplicará polimorfismo al calcular el salario de cada trabajador con el método calcularSalario(), cuya implementación será diferente en cada subclase. Además, se utilizará encapsulación para proteger los atributos y controlar el acceso a los datos.

2. Diseño (Diagrama UML)\

El diseño utiliza herencia mediante la clase abstracta TrabajadorMedico y sus cinco subclases, cada una implementa su propio cálculo salarial, aplicando polimorfismo y las citas se relacionan con un trabajador y conservan un historial de reagendamientos. Se aplica MVC para poder separar la interfaz de consola, el controlador y las clases del modelo, los atributos son privados y su acceso se controla mediante métodos públicos.



3. Enlace de Git Hub

https://github.com/ntp90-cmd/Lab-3---Herencia-Tecnolog-a-M-dica.git